Drugi popravni test iz Algoritama i struktura podataka

Prvi praktični test održan 19.02.2026.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
   a. Vodite računa da imate sve spremno što vam je neophodno.
   b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2025_Test1_GrupaB_IT1g2024**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Bubble Sort*

Napisati metodu ***Sort*** algoritma ***BubbleSort*** za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- U template-u se nalazi klasa ***RandomArrayGenerator*** koju treba iskoristiti za generisanje niza slučajnih brojeva
- U okviru klase ***BubbleSort*** napisati implementaciju za metodu ***public static void sort***(long[] arr)
- Obezbediti kod u **Main** metodi za testiranje algoritma.

Zadatak 2 – *Suma elemenata nakon prvog pojavljivanja vrednosti veće od proseka*

Implementirati metodu koja u prvom prolazu izračunava prosečnu vrednost svih elemenata u listi. U drugom prolazu treba pronaći **prvi čvor** čija je vrednost strogo veća od tog proseka i vratiti sumu svih elemenata koji se nalaze **posle** tog čvora (ne uključujući njega).

Dodatni zahtevi za rešavanje zadatka:

- Zadatak rešiti u linearnom vremenu **O(n).**
- Ako nijedan element nije veći od proseka ili je taj element poslednji u listi, vratiti 0.
- Zahtevan potpis metode: ***public int sumAfterFirstAboveAverage()***

Primer:

- Input: 1 → 2 → **3** → 10 (Prosek = 4)
- Output: 0 (Prvi veći od proseka je 10. Pošto posle njega nema ničega, output je 0)

- Input: 10 → 2 → 1 → 3 (Prosek = 4);
- Output: 10 (Prvi veći od proseka je 10. Output je 2 + 1 + 3 = 6)

Zadatak 3 – *Selektivni odjeci (The Selective Echo)*

Procesiraš niz zvučnih signala koji se čuvaju u digitalnom baferu. Svaki signal je predstavljen celim brojem. Zbog interferencije, novi signali mogu poništiti ili očistiti prethodne na osnovu svoje parnosti:

- **Poništavanje parova:** Ako je novi signal **paran**, on proverava vrh bafera. Ako je na vrhu takođe **paran** broj, dolazi do interferencije – oba signala se poništavaju i nestaju iz bafera.
- **Neparno čišćenje:** Ako je novi signal **neparan**, on redom uklanja (briše) sve **parne** signale sa vrha bafera dok ne naiđe na prvi neparan signal ili dok se bafer ne isprazni. Nakon toga, taj neparni signal ostaje na vrhu.
- **Stabilnost:** Ako je novi signal **paran**, a na vrhu bafera je **neparan** broj, on se samo dodaje na vrh bez ikakvih promena.

Tvoj cilj: Implementirati metodu koja prima niz signala i vraća finalni sadržaj bafera.

Dodatni zahtevi za rešavanje zadatka:

1. Vremenska složenost: **O(n)**.
2. Implementaciju pisati koristeći odgovarajuću strukturu podataka (Stek) za efikasno upravljanje vrhom bafera.

Primer:

Input 1 (nesortirani niz celih brojeva): [3, 4, 6, 5]

Output 1: [3, 5]

Objašnjenje: **(6 poništava 4 jer su oba parna; 5 zatim seda na 3)**

Input 2 (nesortirani niz celih brojeva): [8, 10, 7, 2, 4, 6]

Output 2: [7, 6]

Objašnjenje: **(10 poništava 8; 7 ulazi; 2 seda na 7; 4 poništava 2; 6 seda na 7 -> [7, 6])**